HEATER Ver1

Admin Portal: Complete Project Guide

What the project is, what we used and why, every folder's purpose,
how the boards, mobile app, firmware and dashboard connect,
and what every dashboard graph shows.

Repository: heatrfan-adminportal · Branch: chandra · Firmware: HEATER_Ver1 (22) / FW 1.8.5

Prepared: 16 October 2026 · Author: R. Chandra kumar

Contents

Contents 2

1. Project description 3

Who uses it . . . 3

Main goals . . . 3

2. What we used and why 4

3. System architecture: how everything connects 5

4. Folder structure and purpose 6

4.1 app/(dashboard)/ - dashboard pages . . . 6

4.2 app/api/ - backend API (route handlers) . . . 7

4.3 components/ - React UI building blocks . . . 7

4.4 lib/ - server logic . . . 8

4.5 Other folders and root files . . . 8

5. How data moves 9

6. Firmware: purpose and use 10

6.1 Firmware files in the project . . . 10

6.2 What the heater firmware does . . . 10

6.3 TEMP-HUM Ver1 board . . . 11

7. Mobile app: purpose and connection 12

8. Firmware update (OTA) end to end 13

9. Database 14

10. The dashboard: what each page does 15

11. Every graph explained 18

Temperature probes vs. limit (Device detail) . . . 18

Total power and total current (Device detail) . . . 18

Humidity (Device detail) . . . 19

Wi-Fi mode / online timeline (Device detail) . . . 19

Energy by channel (Device detail) . . . 20

Heat levels, channel health, by room (Overview) . . . 20

Fleet power (Analytics) . . . 21

Temperature envelope (Analytics) . . . 21

Energy by device, where the energy goes, alert frequency (Analytics) . . . 22

Sensor board temperature and humidity (Temperature & Humidity) . . . 22

Signal strength (Signal) . . . 23

12. Security and key design decisions 24

13. Running the project 24

Glossary . . . 25

1. Project description

HEATER Ver1 Admin Portal is the web system that monitors and controls a fleet of smart room heaters. Each heater is an **ESP32-WROOM-32** controller board that switches **four heater elements, a fan and a light** through six relays, measures the current of every output with a CT (current transformer) sensor, and reads temperature and humidity probes. A second, measure-only board, **TEMP-HUM Ver1**, reports four temperature probes and two humidity sensors.

The boards connect over Wi-Fi to the portal's HTTP API and send a telemetry frame about every 15 seconds. The portal stores every value in a **PostgreSQL** database called `heaterfan`, raises alerts (over-current, no-load, over-temperature, offline), rolls up energy per hour, and streams changes live to the admin dashboard. Operators can switch heat levels, change protection limits, register and claim boards, and push new firmware over the air (OTA). A **mobile app** sets up each board over Bluetooth / hotspot, gives it the home Wi-Fi details, controls it locally and can download firmware from the portal.

Who uses it

User	What they do
Admin / operator	Uses the web dashboard: watches the fleet, reads alerts and history, sends commands, edits limits, uploads and rolls out firmware.
Home user	Uses the mobile app: first-time Wi-Fi setup, on/off and heat level, firmware update from the phone.
Heater / sensor board	Registers itself, posts telemetry and heartbeats, collects queued commands, downloads firmware.
Developer	Edits the Arduino firmware (.ino), builds the .bin, maintains the Next.js portal and the database schema.

Main goals

- **Real data only.** Everything on screen is a value stored in the database. If the firmware does not report it, the dashboard does not show it.
- **Live.** The dashboard updates by push (Server-Sent Events); nothing polls the page.
- **Safe.** Protection limits, alerts that clear only from evidence, the fan-is-master rule in firmware, and SHA-256 checked firmware updates.
- **Survives restarts.** Nothing important is held only in server memory; online/offline is derived from the last-seen time.

2. What we used and why

Technology	Used for	Why we chose it
Next.js 16 (App Router)	Web pages and the HTTP API in one project	One codebase and one server for both the dashboard and the device API; server components read the database directly.
React 19	Dashboard UI	Component model for tables, cards and charts that update live.
TypeScript	All portal code	Types in lib/types.ts mirror the database exactly, so mistakes show at build time.
Tailwind CSS v4	Styling	Fast, consistent styling without separate CSS files.
PostgreSQL + pg driver	All storage	Reliable relational storage, transactions, partial unique indexes for alerts, FOR UPDATE SKIP LOCKED for command queues.
Server-Sent Events (SSE)	Live updates to the browser	Traffic is one-way (server to browser); SSE reconnects automatically and needs no second server. Commands travel as normal POSTs.
Hand-written SVG charts	components/charts.tsx	No chart library dependency; full control over thinning and live updates.
ESP32-WROOM-32 + Arduino	Heater and sensor firmware	Built-in Wi-Fi and Bluetooth, enough ADC pins for 6 CT sensors, OTA partitions.
DS18B20 / DHT22 sensors	Temperature and humidity	Cheap, digital, one pin each; DHT22 also gives humidity.
SCT-013 style CT sensors	Current per output	Non-invasive current measurement to detect no-load, over-current and to compute power.
Bluetooth (SPP/BLE) + hotspot	Mobile app setup	Lets the phone configure Wi-Fi before the board is on the network, and act as a gateway.
SHA-256	Device keys and firmware	Device keys are stored only as digests; firmware images are verified before flashing.

3. System architecture: how everything connects

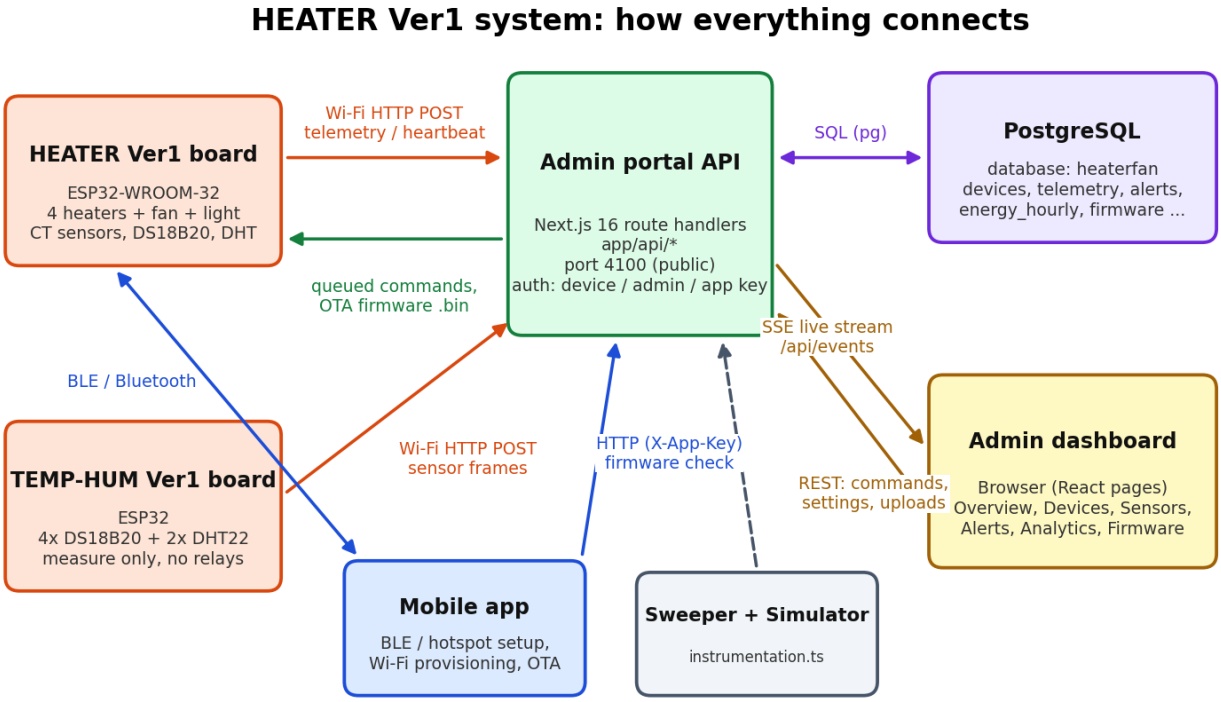
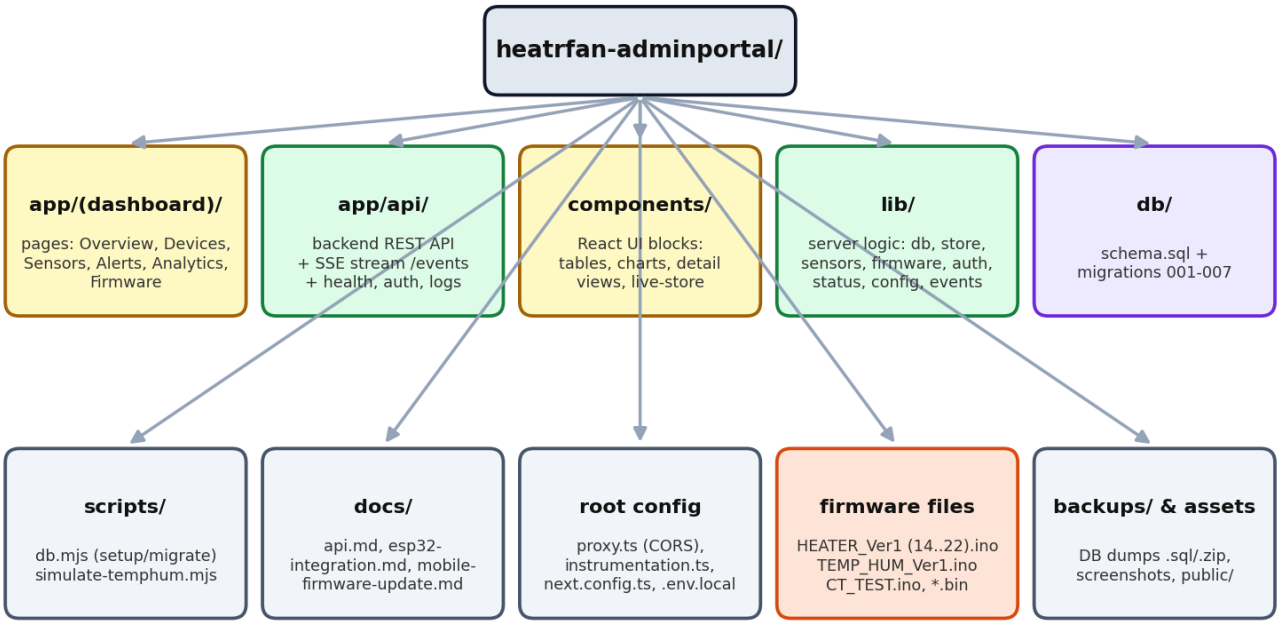


Figure 1. The five parts of the system and the links between them.

Link	From → To	How	Purpose
Telemetry	Board → Portal API	Wi-Fi, HTTP POST /api/devices/{id}/telemetry with x-device-key	Every ~15 s: relays, current, power, temperatures, humidity, Wi-Fi/BLE signal.
Heartbeat	Board → Portal API	HTTP POST /heartbeat	Proof of life when idle, without writing a frame.
Commands	Portal → Board	Returned inside the telemetry / heartbeat response	Heat level, on/off, settings, FIRMWARE_UPDATE. Board acknowledges with PATCH.
Database	Portal ↔ PostgreSQL	pg connection pool (lib/db.ts)	Store and read everything.
Live stream	Portal → Dashboard	SSE GET /api/events	Push each changed device to every open browser.
Admin actions	Dashboard → Portal	REST calls with the admin cookie / x-admin-key	Send commands, edit limits, upload firmware, acknowledge alerts.
Bluetooth	Mobile app ↔ Board	Bluetooth SPP / BLE or the board's hotspot	Wi-Fi setup, local control; app can forward frames as a gateway.
App API	Mobile app → Portal	HTTP with X-App-Key	Check current firmware and download the .bin.

The portal is published on `http://50.186.180.197:4100` (and `asvrladmin.mo.vc`); the heater firmware has this address built in as `CLOUD_SERVER_URL`. When the server starts, `instrumentation.ts` checks the database, starts the **sweeper** (marks silent devices offline and prunes old history) and, only if `HEATER_SIM=on`, the virtual-fleet **simulator**.

4. Folder structure and purpose



Top row = runs inside the portal. Bottom row = tooling, docs, firmware and assets.

Figure 2. Project folders. The top row runs inside the portal; the bottom row is tooling, docs and firmware.

4.1 app/(dashboard)/ - dashboard pages

Path	Page	Purpose
page.tsx	Overview /	Fleet summary tiles, needs-attention list, live activity feed, heat-level and channel-health charts, by-room breakdown, channel wall.
devices/page.tsx	Devices /devices	Searchable, filterable, sortable list of all heaters; add a device.
devices/[id]/page.tsx	Device detail	Everything about one heater: live cards, channels, history charts, alerts, commands, audit trail.
sensors/, sensors/[id]/	Temperature & Humidity	List and detail of TEMP-HUM boards: latest readings, history charts, alert limits, CSV export.
alerts/page.tsx	Alerts	Fleet fault log, open or resolved, filter by severity, acknowledge.
analytics/page.tsx	Analytics	Historical energy, temperature envelope, energy by device and channel type, alert frequency.
firmware/page.tsx	Firmware	Upload .bin releases, publish, mark current, roll out to the fleet.
layout.tsx, error.tsx, not-found.tsx	Shell	Sidebar + header around every page, error and not-found screens.
app/login/page.tsx	Login	Admin sign-in (sets the heater_admin cookie).

4.2 app/api/ - backend API (route handlers)

Route	Caller	Purpose
devices (POST / GET)	board / admin	Register a board (returns its key) / list devices.
devices/add, devices/claim	admin / app	Pre-add a serial on the portal; claim a board into a home and room.
devices/[id]	admin	Device detail, rename / move, delete.
devices/[id]/telemetry	board	Post one frame; response carries queued commands.
devices/[id]/heartbeat	board	Proof of life.
devices/[id]/commands	both	Queue (admin), claim and acknowledge (board).
devices/[id]/settings	admin	Protection limits and climate target.
devices/[id]/history, energy, alerts	admin	Time-series history, hourly energy, device alerts.
devices/[id]/pairing	admin / app	Pairing / provisioning details.
devices/[id]/firmware(/download)	board	Is there an update for me? / download the .bin.
firmware, firmware/[id], [id]/rollout, [id]/download	admin	Upload, edit, publish, delete, roll out, download releases.
firmware/current(/download)	mobile app	Current version check and image download (X-App-Key).
sensors/… (latest, history, settings, alerts, export.csv)	admin	TEMP-HUM board data and settings.
events	dashboard	SSE live stream.
alerts, rooms, stats, logs	admin	Fleet alerts, rooms, counters, API request log.
auth/login, auth/logout	browser	Sign in / out.
health	anyone	Database and schema check.

4.3 components/ - React UI building blocks

File	Purpose
shell.tsx	Sidebar navigation, header, Live badge, sign out, monitoring thresholds panel.
live-store.tsx	Opens the SSE stream and keeps the in-browser copy of the fleet that every view reads.
fleet-summary.tsx, overview-panels.tsx	Overview tiles, needs attention, activity feed, heat levels, channel health, by room, channel wall.
device-table.tsx, add-device.tsx	Devices list (search / filter / sort / paginate) and the Add-device dialog.
device-detail.tsx	Device page: live cards, channel table, connectivity, history charts, alerts, commands, audit trail.
charts.tsx	Dependency-free SVG charts: TimeSeriesChart, DutyChart, TimelineStrip.
sensor-list.tsx, sensor-detail.tsx, sensor-chart.tsx, sensor-shared.tsx	TEMP-HUM pages and their charts.
alerts-view.tsx, analytics-view.tsx, firmware-view.tsx	Alerts, Analytics and Firmware pages.
status.tsx, login-form.tsx, ui/primitives.tsx	Status badges, the login form, basic Card / Badge / Button pieces.

4.4 lib/ - server logic

File	Purpose
db.ts	PostgreSQL connection pool, number parsing, transactions, health check.
store.ts	Main repository: ingest telemetry, queries, alert raise/clear, energy rollup, commands, sweeper.
sensors.ts / sensor-types.ts	Same for TEMP-HUM boards.
firmware.ts	Firmware releases, SHA-256, publish / current / rollout.
auth.ts	Checks device key, admin key / cookie, app key, provisioning key.
status.ts	Derives online / offline / stale and signal strength buckets.
config.ts	Reads every HEATER_* setting from .env (timeouts, limits, RSSI bands, keys).
events.ts	In-process EventBus that feeds the SSE stream.
clock.ts, format.ts	Time windows (Today / 24 h / 7 d / 30 d) and value formatting in Asia/Kolkata.
api.ts, api-log.ts	Shared API helpers and the request log (api_requests table).
simulator.ts	Virtual heater fleet for demos (HEATER_SIM=on).
types.ts	Domain model that mirrors the database schema.

4.5 Other folders and root files

Item	Purpose
db/schema.sql	Complete database structure; rebuilds the database on any machine.
db/migrations/001-007	Ordered changes: 001 signal strength, 002 firmware releases, 003 API request log, 004 fourth temperature probe, 005 device claims, 006 temp/hum sensors, 007 device name source.
scripts/db.mjs	node scripts/db.mjs setup | status | migrate.
scripts/simulate-temphum.mjs	Sends fake TEMP-HUM sensor frames for testing.
docs/api.md	Full HTTP API reference.
docs/esp32-integration.md	Contract for firmware: register, telemetry, heartbeat, Bluetooth, OTA.
docs/mobile-firmware-update.md	Mobile app guide for firmware check and download.
proxy.ts	CORS for every /api route (HEATER_CORS_ORIGINS).
instrumentation.ts	Startup: database check, sweeper, simulator.
next.config.ts	Turbopack root and allowed dev hosts (asvrladmin.mo.vc, 50.186.180.197).
.env.local	Database URL, admin key and other secrets / settings (never commit).
backups/	Database dumps (heaterfan-20261015-0716.sql / .zip).
public/, image.png, Screenshot (49-52).png	Static icons and screenshots (not used by the running app).
.next/, node_modules/, tsconfig.tsbuildinfo	Generated by build / install. Do not edit.
.playwright-mcp/	Logs from automated browser checks. Safe to delete.

5. How data moves

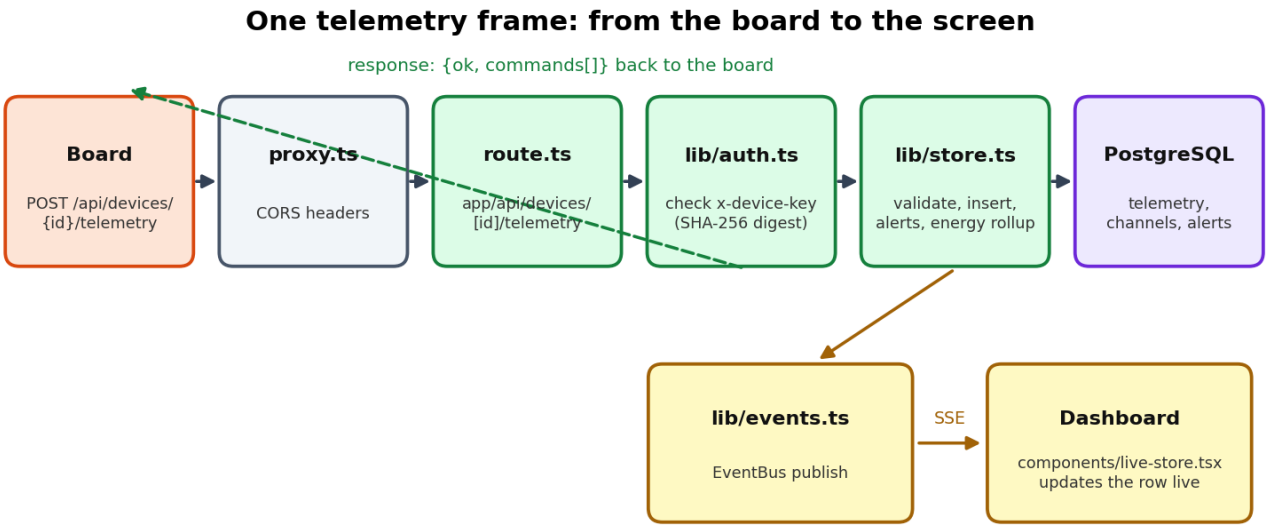


Figure 3. One telemetry frame passing through the code.

- **proxy.ts** adds CORS headers before any handler runs.
- **auth.ts** hashes the x-device-key and compares it with the stored SHA-256 digest.
- **store.ts** validates the frame (bad fields get HTTP 422), writes `telemetry` and `telemetry_channels`, raises or clears alerts, adds energy to `energy_hourly`.
- **events.ts** publishes the change; every open dashboard receives it over SSE and **live-store.tsx** updates that one row.
- The HTTP response returns any pending commands to the board.

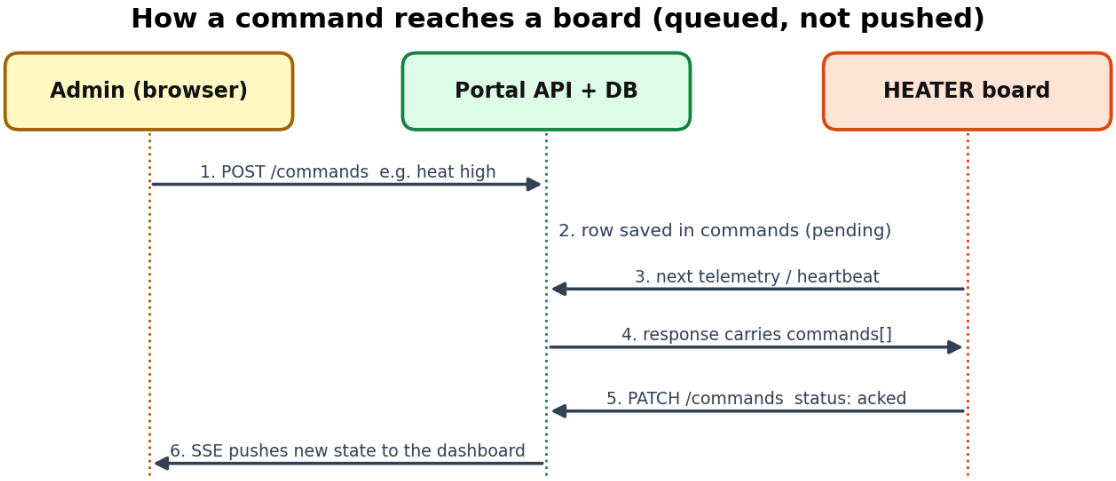


Figure 4. Commands are queued in the database and collected by the board on its next round trip.

Two boards polling at once can never collect the same command, because the claim uses `FOR UPDATE SKIP LOCKED`. A command the board never collects expires.

6. Firmware: purpose and use

Firmware is the Arduino (C++) program running on the ESP32 board. It switches the relays, reads the sensors, protects the hardware, serves a small local web page, talks Bluetooth to the mobile app, and reports to the portal. The portal never runs this code; the files are kept in the project folder for reference and to build OTA images.

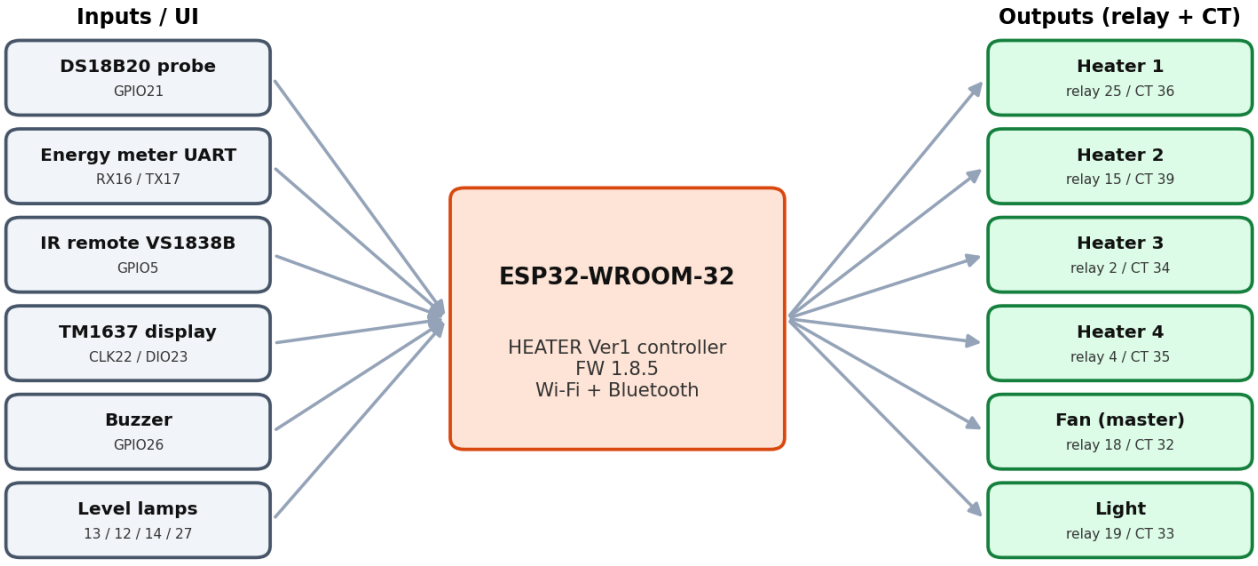


Figure 5. HEATER Ver1 board: inputs, outputs and pins (from the HEATER_Ver1 (22).ino header).

6.1 Firmware files in the project

File	What it is	Use
HEATER_Ver1 (22).ino	Latest heater controller firmware, FW 1.8.5	Current source. Built into HEATER_Ver1_22_fw1.8.5.bin.
HEATER_Ver1 (14) … (21).ino	Earlier heater firmware versions	History; (14)/(15) were the first portal-linked builds, later ones added OTA, Wi-Fi scan, fan-master rule, etc.
HEATER_Ver1_NN_fwX.Y.Z.bin	Compiled firmware images (1.4.0 … 1.8.5)	Uploaded on the Firmware page for OTA updates.
nirmasl final board (copy)	Older full controller firmware	Reference / backup only.
TEMP_HUM_Ver1.ino	TEMP-HUM sensor board firmware	4x DS18B20 + 2x DHT22, no relays; appears under /sensors.
CT_TEST.ino	Standalone current-sensor test	Bench test: prints the ADC midpoint and current of each CT once a second.

6.2 What the heater firmware does

- **Heat levels:** OFF = no heater, LOW = H1, MEDIUM = H1+H2, HIGH = H1+H2+H3+H4. Fan and light are separate on/off outputs.
- **Fan is master:** a heater can only be on while the fan is on. Fan on → 2 s → heaters on; heaters off → 2 s → fan off. A safety tick drops any heater if the fan is off.
- **Auto-comfort (climate):** keeps a target temperature; on by default; target saved in flash.

- **Protection:** per-channel current limit, total current limit and temperature limit; CT readings detect no-load and over-current.
- **Four control channels, same commands:** Serial (115200), Bluetooth name HEATER_VER1, local web page http://<ip>/ or heater.local, and a JSON API (/api/status, /api/cmd, /api/wifiscan) protected by an API key.
- **Also:** IR remote (learnable keys), 4-digit TM1637 display, buzzer, level lamps, optional energy meter on UART.
- **Cloud link:** registers with the portal, posts telemetry, collects commands, checks `/api/firmware/current` and performs OTA.

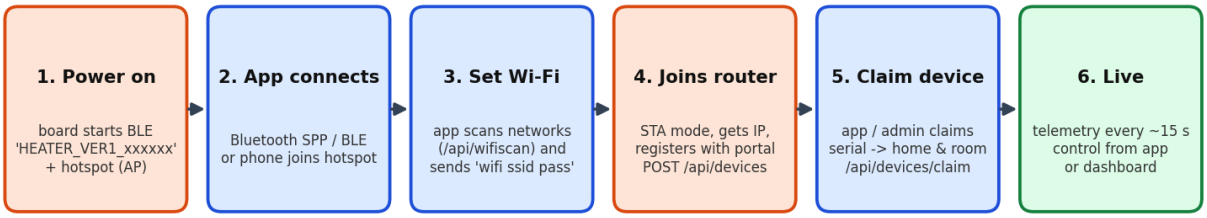
6.3 TEMP-HUM Ver1 board

Measure-only board: T1-T4 DS18B20 probes on GPIO 21, 22, 18, 19 and two DHT22 humidity sensors on GPIO 4 and 23. To the mobile app it looks exactly like a heater (same Bluetooth name format and status JSON) so the same app can set its Wi-Fi. To the portal it registers as model **TEMP-HUM Ver1** with serial `TH-VER1-<id>`, so it is shown on the Temperature & Humidity pages.

7. Mobile app: purpose and connection

The mobile app is the home user's tool. It is a separate project (not in this repository), but it talks to both the board and this portal.

Mobile app: first-time setup of a board



Blue = done in the mobile app. Orange = board. Green = portal. If Wi-Fi is down, the app can act as a BLE gateway and post the frame for the board.

Figure 6. First-time setup of a board with the mobile app.

What the app does	How
Find and connect to a new board	Bluetooth (name HEATER_VER1_<chip id>) or by joining the board's hotspot.
Give the board the home Wi-Fi	Reads nearby networks from /api/wifiscan on the board and sends wifi <ssid> <pass>.
Control the heater	Same command set as the dashboard: heat level, fan, light, climate on/off, set temperature.
Act as a gateway	If the board has no Wi-Fi, the app reads the frame over Bluetooth and posts it to the portal with the board's key (bluetoothClient: true, wifi: AP).
Firmware update	GET /api/firmware/current?model=HEATER%20Ver1&version=<fw> with X-App-Key → if updateAvailable, download the .bin, verify SHA-256, send to the board.

```
GET /api/firmware/current?model=HEATER%20Ver1&version=1.8.3
X-App-Key: <HEATER_APP_KEY>

-> { updateAvailable: true, current: { version: "1.8.5", sha256, sizeBytes, notes },
     downloadUrl: ".../api/firmware/current/download?model=HEATER%20Ver1&version=1.8.5" }
```

8. Firmware update (OTA) end to end

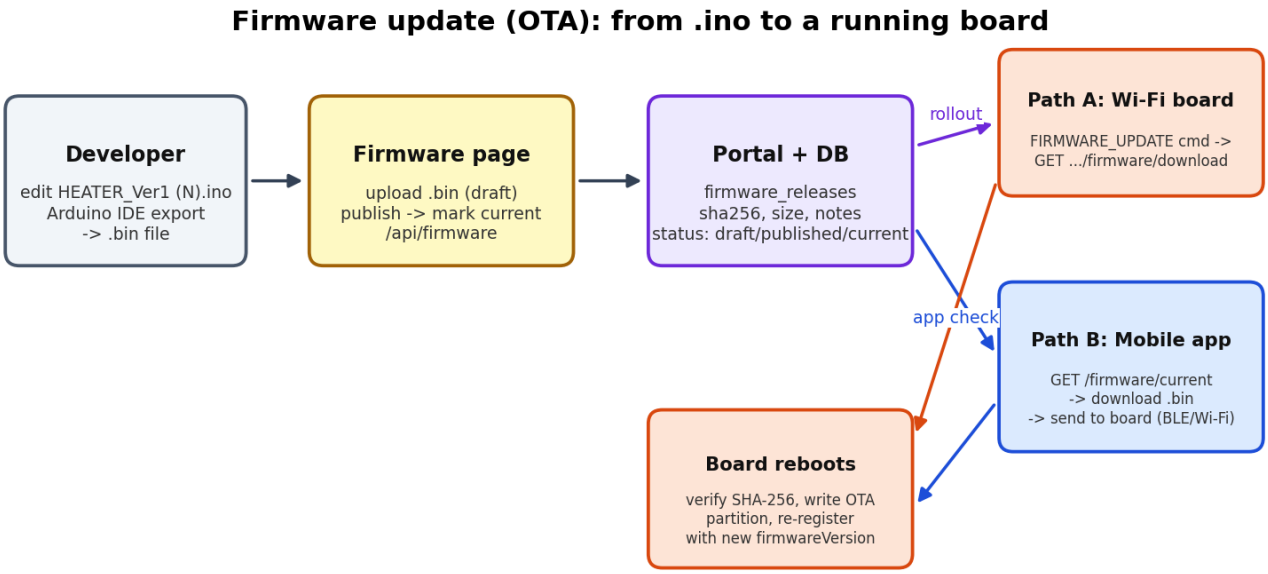


Figure 7. From an .ino file to a board running the new version.

- **Build:** export the compiled binary from Arduino IDE (e.g. HEATER_Ver1_22_fw1.8.5.bin). Bump `FW_VERSION` on every release.
- **Upload:** Firmware page → upload as a draft. The portal stores size and SHA-256 in `firmware_releases`.
- **Publish and mark current:** drafts are never downloadable; only published releases are.
- **Path A, rollout:** the portal queues `FIRMWARE_UPDATE 1.8.5` for each outdated board; the board downloads from /api/devices/{id}/firmware/download.
- **Path B, mobile:** the app checks /api/firmware/current and downloads the image.
- **Board:** writes the inactive OTA partition, checks SHA-256, acknowledges, reboots and re-registers with the new version. The Firmware page's adoption figures come from that reported version.

9. Database

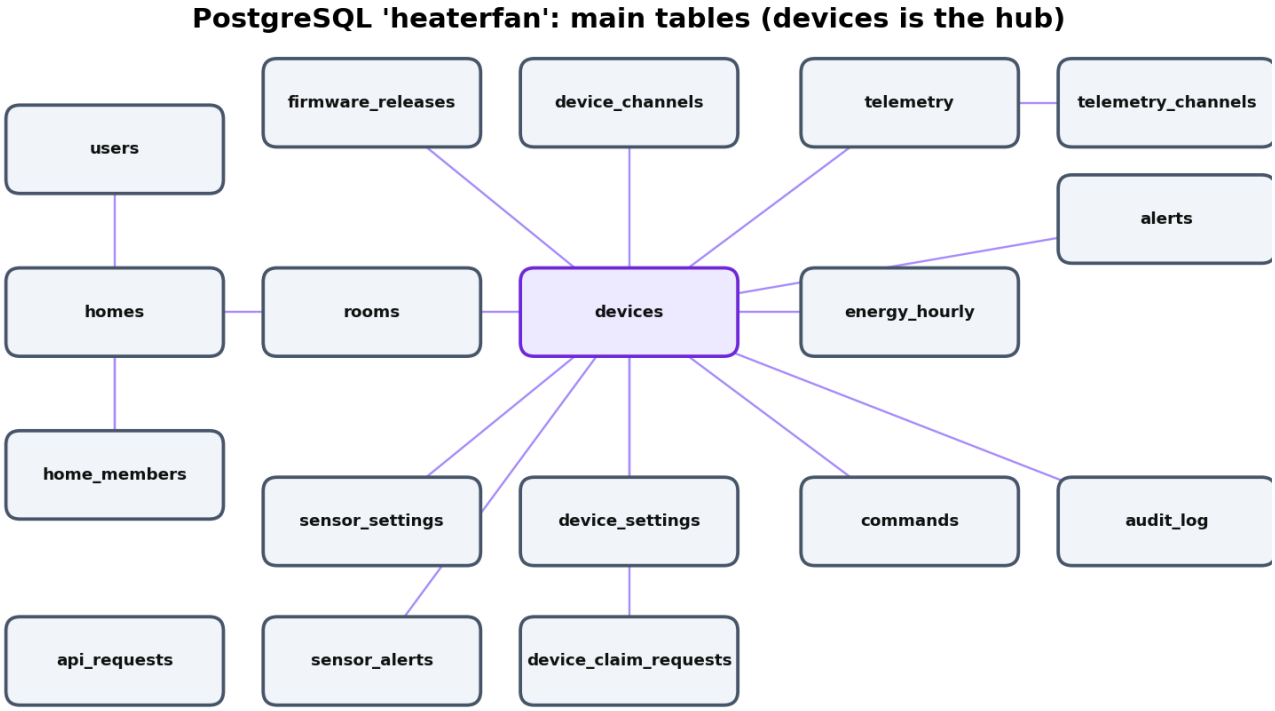


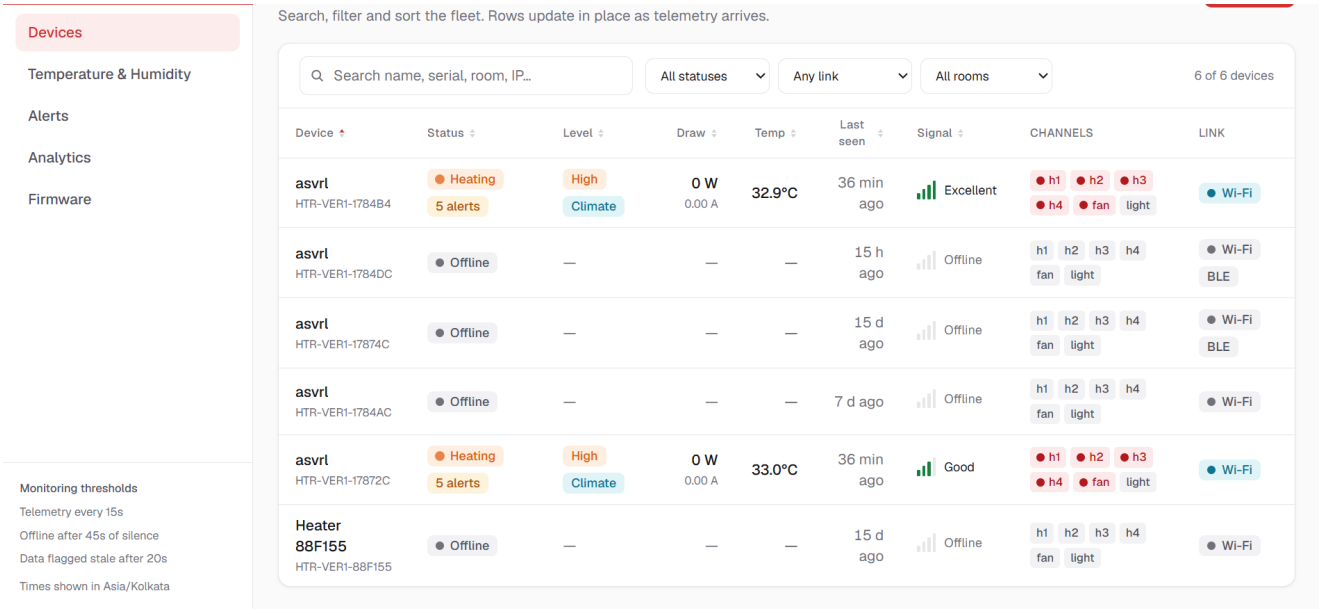
Figure 8. Main tables of the heaterfan database.

Table	Holds
devices	One row per board: serial, model, name, room, firmware version, last seen, key digest, Wi-Fi/BLE state.
device_channels	Six outputs per heater: type, relay and CT pins, calibration.
device_settings	Protection limits and climate target per device.
telemetry / telemetry_channels	Every frame and every channel reading in it (the largest tables).
energy_hourly	Energy integrated per hour per channel, used by charts and analytics.
alerts	Raised / cleared faults; one open row per (device, channel, code).
commands	Queued commands and their status (pending, delivered, acked, failed, expired).
audit_log	Configuration changes made against each device.
firmware_releases	Uploaded firmware images, SHA-256, status and notes.
users, homes, home_members, rooms	Ownership: who owns which home and the rooms in it.
device_claim_requests	Boards added or claimed before they first report.
sensor_settings, sensor_alerts	Limits and alerts for TEMP-HUM boards.
api_requests	Request log shown by /api/logs.
schema_migrations	Which migration files are applied.

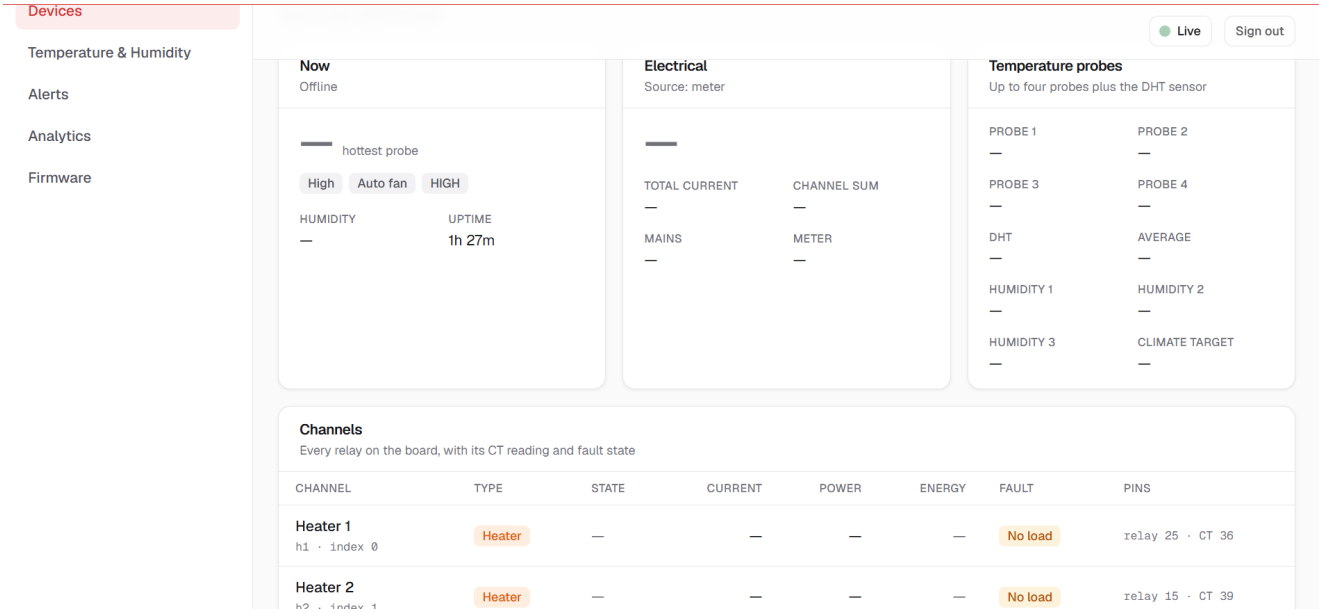
10. The dashboard: what each page does

Page	What you see	What you can do
Overview	Devices, online, heating, fleet draw, hottest probe, open alerts, channel faults, connectivity; needs-attention list; live activity; charts.	Click a tile to jump to filtered devices.
Devices	Each heater with status, heat level, power, temperature, last seen, signal, six channel chips (h1-h4, fan, light), Wi-Fi/BLE link.	Search, filter by status / link / room, sort, add a device, open detail.
Device detail	Now / Electrical / Temperature cards, channel table with current, power, energy, fault and pins, connectivity, protection limits, history charts.	Send commands, change limits, rename / move, acknowledge alerts, pick Today / 24 h / 7 d / 30 d.
Temperature & Humidity	TEMP-HUM boards with latest probes and humidity.	History charts, alert limits, sensor names, CSV export.
Alerts	Open and resolved faults across the fleet.	Filter by severity, acknowledge.
Analytics	Energy and temperature over time, energy split, alert frequency.	Choose the time range.
Firmware	Every release, which one is current, how many boards run each version.	Upload, publish, mark current, roll out, download.

Status rules: a board is online if it reported within the offline timeout (45 s on this install); data older than 20 s is marked "not live"; offline boards show — instead of old values, so a silent heater is never drawn as running. Times are shown in Asia/Kolkata.



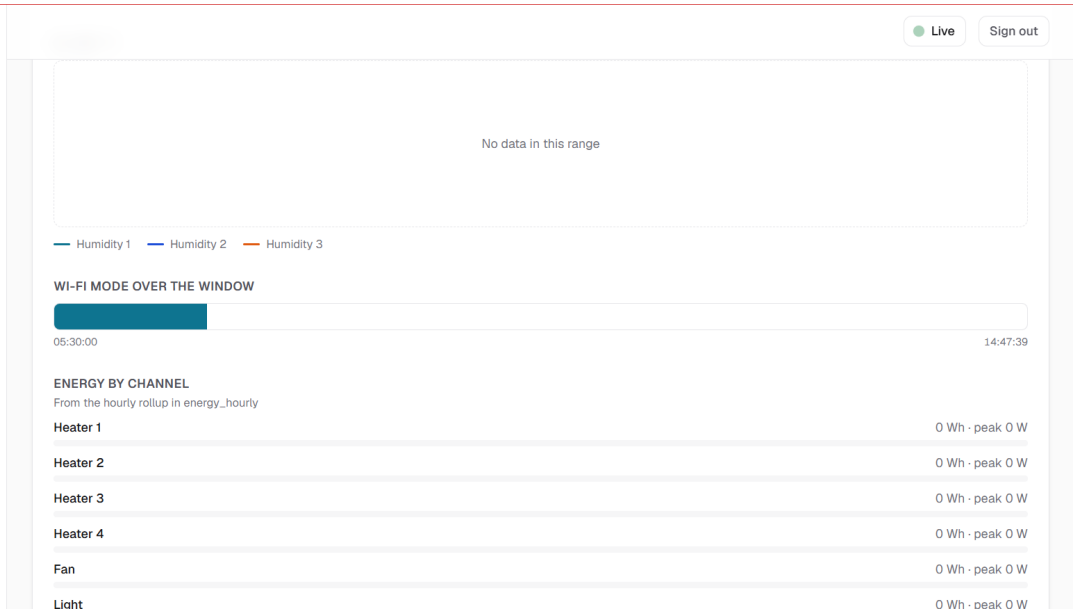
Screenshot. Devices page on asvrladmin.mo.vc: two heaters heating on High with Climate, others offline.



Screenshot. Device detail: Now, Electrical and Temperature probe cards, and the Channels table with relay and CT pins.



Screenshot. Device detail history: Temperature probes vs. limit (85 °C), Total power and Total current.

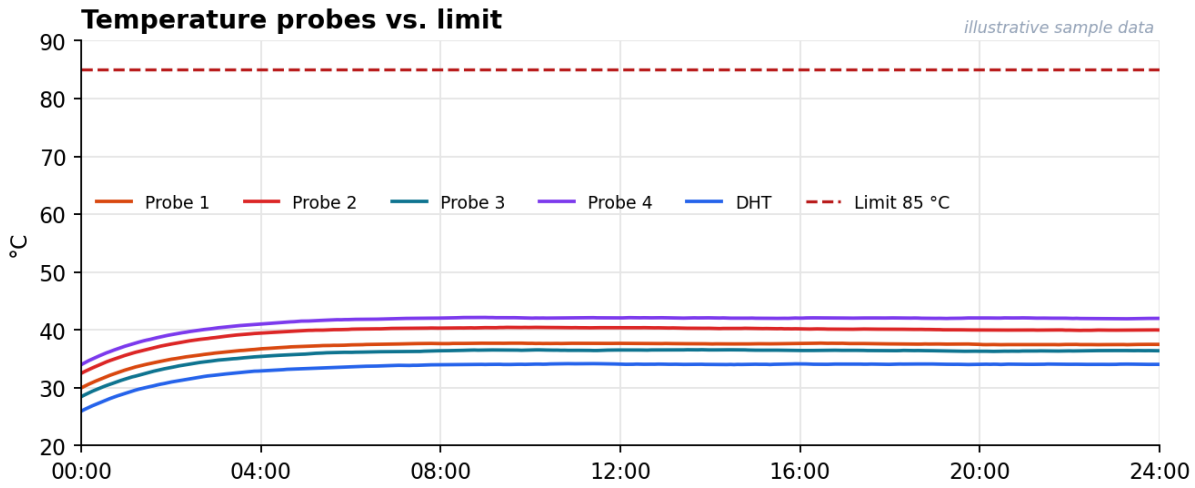


Screenshot. Device detail: Humidity chart, Wi-Fi mode over the window and Energy by channel.

11. Every graph explained

The pictures below show the shape of each graph on the dashboard. They are drawn with **illustrative sample data** so every graph is visible; the real dashboard draws the same graphs from your database. Charts thin long ranges by picking real readings (never averaging), so a short spike is never hidden.

Temperature probes vs. limit (Device detail)

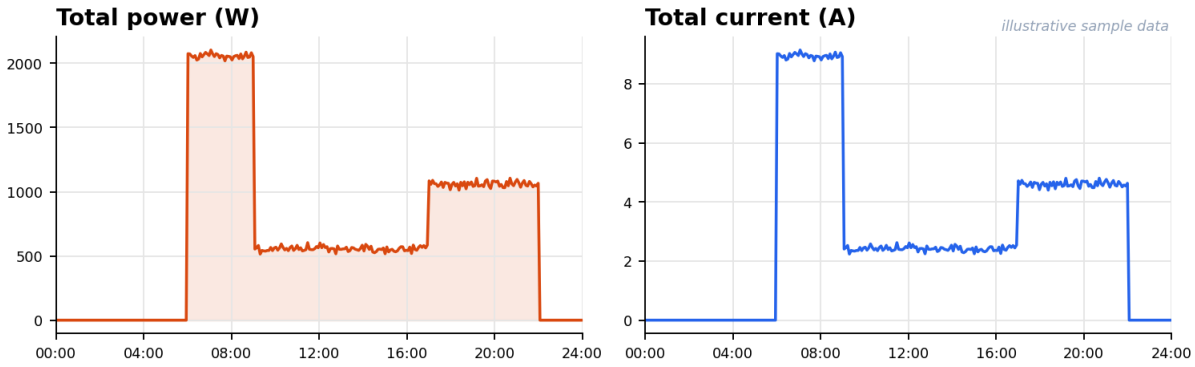


Temperature probes vs. limit: illustrative sample data

Shows: Probes 1-4 and the DHT sensor over the chosen window, with the device's temperature limit as a dashed red line.

Why it matters: See whether the heater is warming normally and how close it gets to the cut-off. A probe that stays flat or shows no data points to a wiring or sensor fault.

Total power and total current (Device detail)

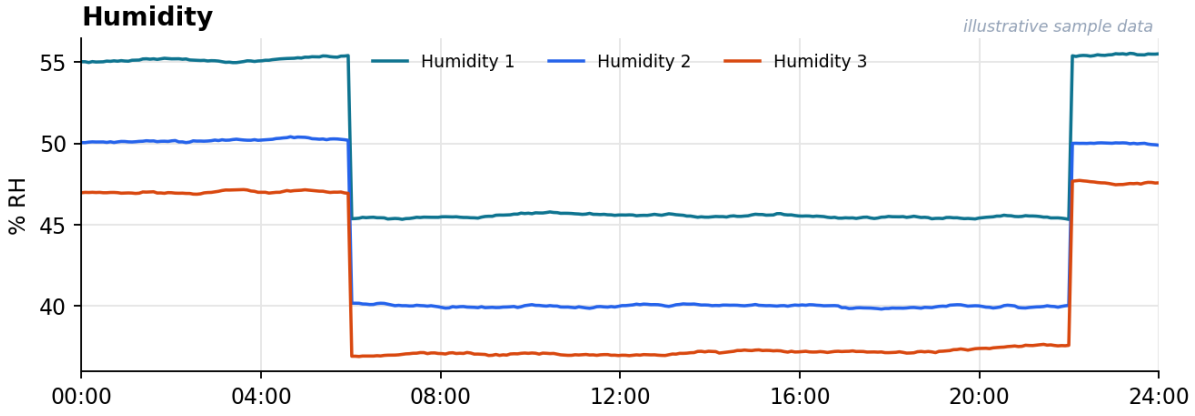


Total power and total current: illustrative sample data

Shows: Watts and amps drawn by the whole board over time, from the CT sensors or the energy meter.

Why it matters: Steps match heat levels (LOW ≈ 1 element, HIGH ≈ 4). A heat level with zero power means a no-load fault or a failed element.

Humidity (Device detail)

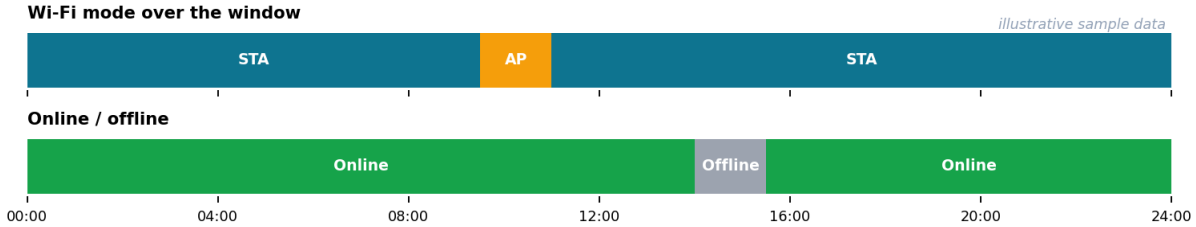


Humidity: illustrative sample data

Shows: Humidity 1-3 in % RH.

Why it matters: Heating dries the room; this shows comfort and confirms the humidity sensors work.

Wi-Fi mode / online timeline (Device detail)

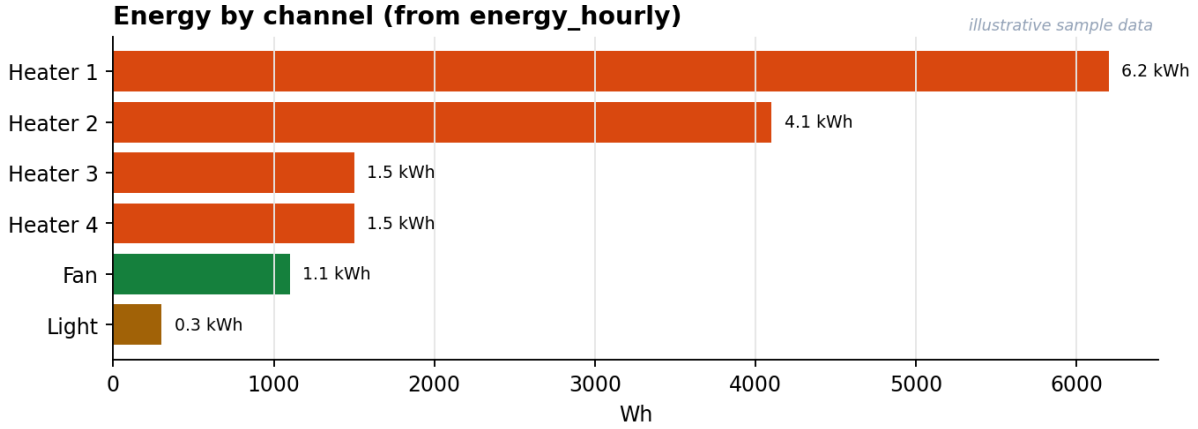


Wi-Fi mode / online timeline: illustrative sample data

Shows: A strip showing when the board was on the home network (STA), in hotspot mode (AP) or offline.

Why it matters: Explains gaps in the other charts and shows unstable Wi-Fi.

Energy by channel (Device detail)

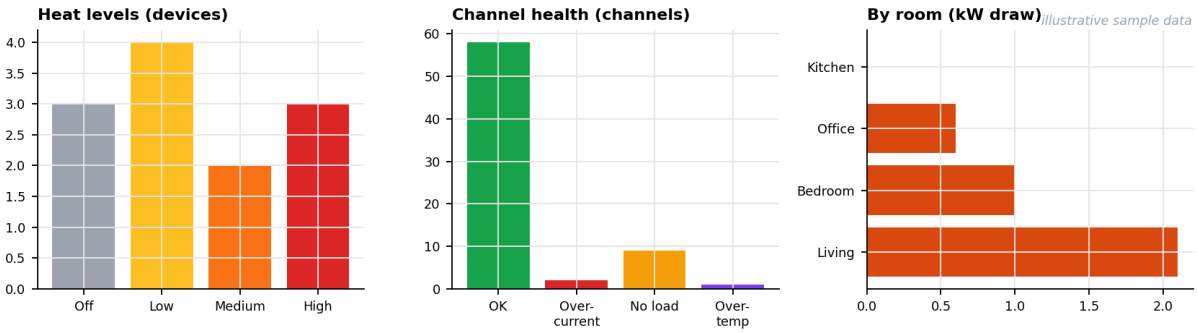


Energy by channel: illustrative sample data

Shows: Energy used by each output (Heater 1-4, Fan, Light) in the window, from energy_hourly, with peak power.

Why it matters: Shows which elements do the most work. Heater 1 is always highest because every level uses it.

Heat levels, channel health, by room (Overview)

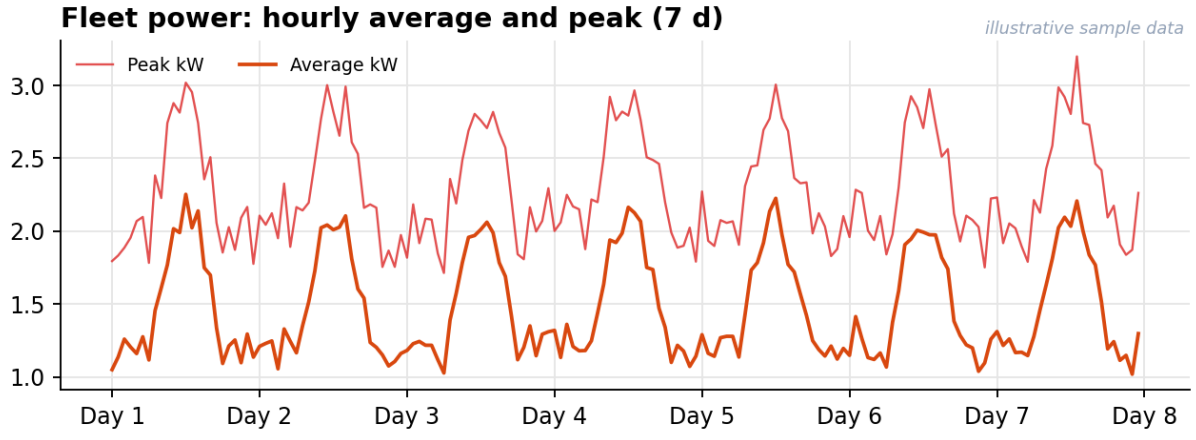


Heat levels, channel health, by room: illustrative sample data

Shows: How many heaters are at each level; how many channels are OK / over-current / no-load / over-temperature; current draw per room.

Why it matters: A one-glance view of the fleet's state and where faults or load are concentrated.

Fleet power (Analytics)

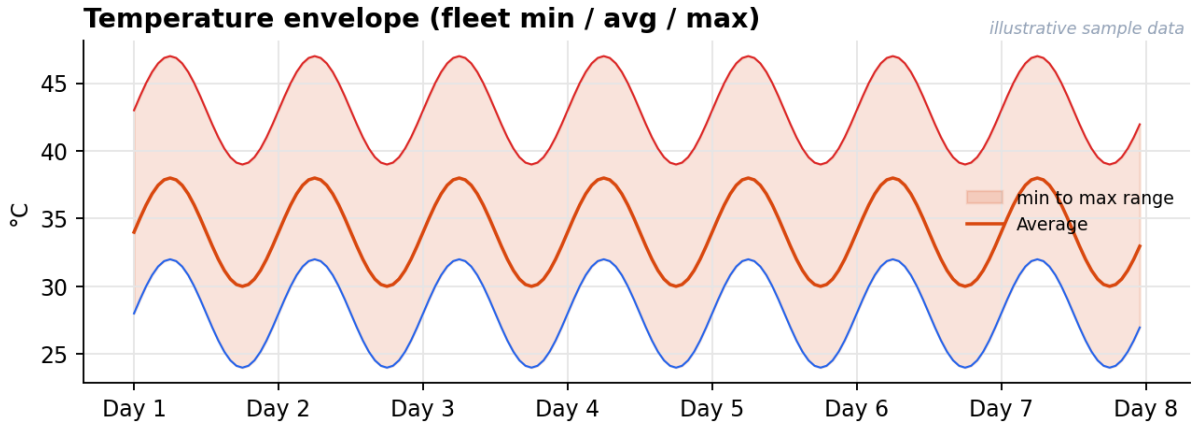


Fleet power: illustrative sample data

Shows: Hourly average and peak power of the whole fleet.

Why it matters: Daily patterns, peak demand, and how much capacity the supply needs.

Temperature envelope (Analytics)

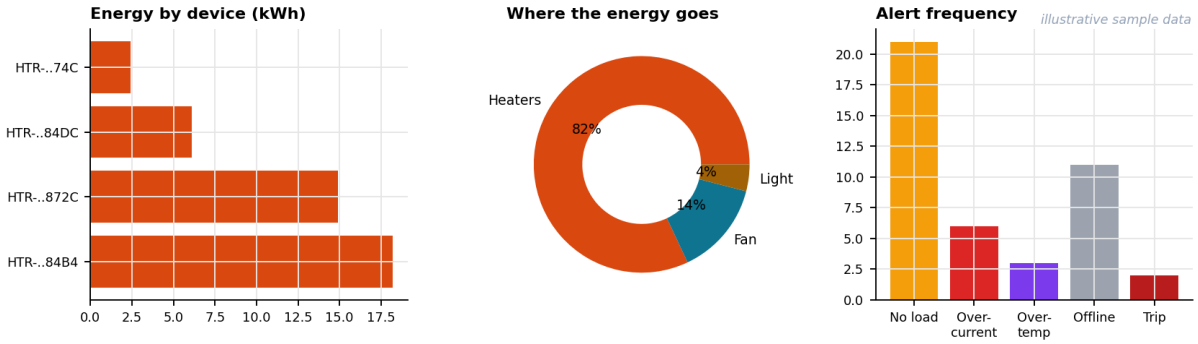


Temperature envelope: illustrative sample data

Shows: Lowest, average and highest probe temperature across the fleet per hour.

Why it matters: Spots units running too hot or rooms that never warm up.

Energy by device, where the energy goes, alert frequency (Analytics)

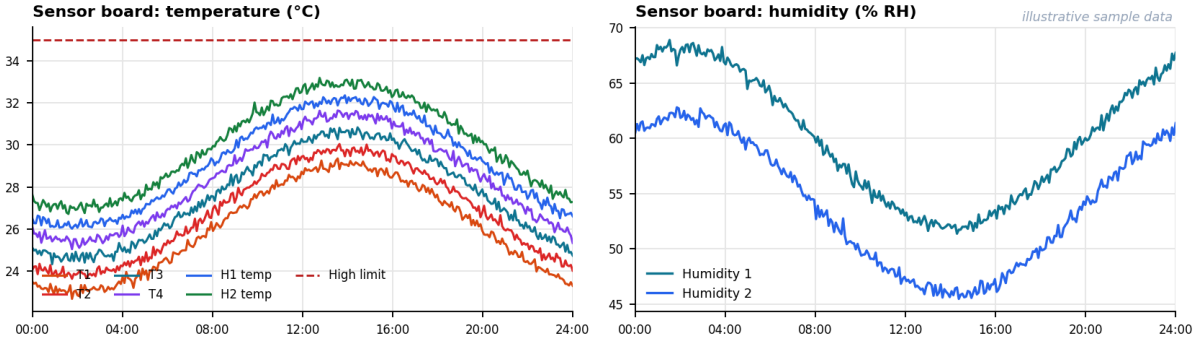


Energy by device, where the energy goes, alert frequency: illustrative sample data

Shows: kWh per device, share of energy by channel type (heaters / fan / light), and how often each alert type happens.

Why it matters: Find the most expensive units and the most common faults to fix first.

Sensor board temperature and humidity (Temperature & Humidity)

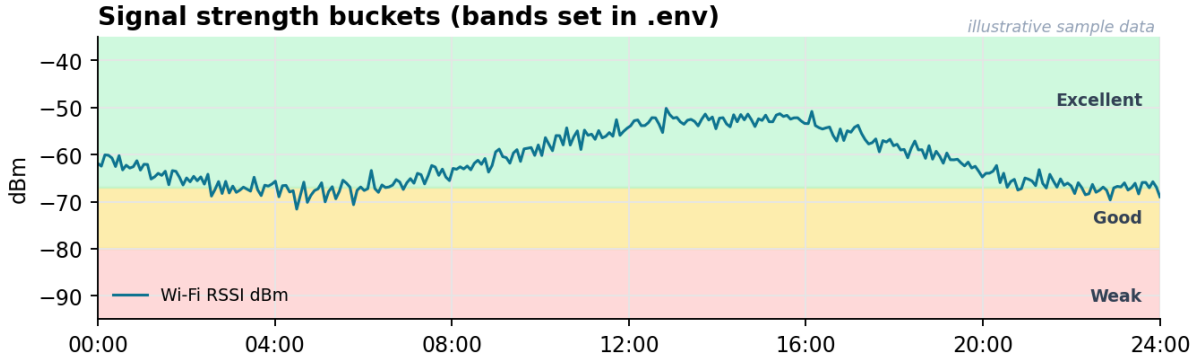


Sensor board temperature and humidity: illustrative sample data

Shows: T1-T4 probes and the two DHT22 temperatures; Humidity 1 and 2. Limits are set per sensor.

Why it matters: Room monitoring; readings outside the limits raise sensor alerts. Data can be exported as CSV.

Signal strength (Signal)



Signal strength: illustrative sample data

Shows: Measured Wi-Fi (and BLE) RSSI in dBm, bucketed Excellent / Good / Weak using the bands in .env.

Why it matters: A weak link explains missed frames. A link that is up but not measured is shown as "Not reported", never guessed.

12. Security and key design decisions

Decision	Reason
Device keys stored as SHA-256 digests	A copy of the devices table cannot be used to pretend to be a heater. The plain key is shown once at registration.
Three kinds of key	x-device-key (boards), admin key / heater_admin cookie (dashboard), X-App-Key (mobile app). HEATER_PROVISIONING_KEY is the secret firmware presents on ingest.
Status is derived, not stored	Online is computed from last_seen_at every time, so it can never go stale.
Alerts clear from evidence	An alert closes only when a frame arrives without the fault; acknowledging just records that someone looked.
Energy is integrated	Power × time since the previous frame, capped so an outage cannot book energy never used. Reboots do not cause negative jumps.
Bad data rejected	Malformed frames get HTTP 422 naming the field, so history stays clean.
Firmware verified	SHA-256 of every image is checked before flashing; drafts can never be downloaded.
Simulator off by default	HEATER_SIM drives every device, including real ones, so it must stay off once hardware is reporting.

13. Running the project

```
npm install
createdb heaterfan
node scripts/db.mjs setup      # apply db/schema.sql
node scripts/db.mjs status     # applied / pending migrations
node scripts/db.mjs migrate    # apply new migrations
npm run dev                    # start the portal
HEATER_SIM=on npm run dev      # demo fleet (never with real boards)
npm run build && npm start     # production
```

Setting (.env.local)	Meaning
DATABASE_URL	postgresql://heater@localhost:5432/heaterfan by default.
HEATER_ADMIN_KEY	Password for the dashboard and admin API.
HEATER_APP_KEY	Key the mobile app sends as X-App-Key.
HEATER_PROVISIONING_KEY	Secret the firmware presents on ingest (unset = open, development only).
HEATER_OFFLINE_TIMEOUT_MS / HEATER_STALE_DATA_MS	When a board counts as offline / its data as not live.
HEATER_DEFAULT_*_LIMIT	Default current, total-current and temperature limits for new devices.
HEATER_WIFI_RSSI_* / HEATER_BLE_RSSI_*	Signal strength bands.
HEATER_HISTORY_RETENTION_MS	How long telemetry history is kept.
HEATER_TIMEZONE	Display time zone (Asia/Kolkata).
HEATER_CORS_ORIGINS	Which browser origins may call the API.

Glossary

Term	Meaning
ESP32	Microcontroller with Wi-Fi and Bluetooth used on both boards.
CT	Current transformer: clamps around a wire and measures current.
Telemetry frame	One JSON report from a board with all live values.
SSE	Server-Sent Events: a one-way live stream from server to browser.
OTA	Over-the-air firmware update.
STA / AP	Wi-Fi modes: STA = joined the home router, AP = board runs its own hotspot.
RSSI	Received signal strength in dBm (closer to 0 is stronger).